

data_organization_example

Tổ chức dữ liệu Train / Validation / Test — ví dụ cụ thể

Tài liệu này mô tả bằng ví dụ có thật cách dữ liệu được chia và ghép cho ba luồng của mô hình (LSTM giá, GNN đồ thị, News), nhằm làm rõ hai điểm: (1) **không trộn ngày chéo mã** trong một snapshot, và (2) **không rò rỉ (leakage)** giữa các tập train/validation/test.

Số liệu biên split trong tài liệu là **tính trực tiếp** từ `data/processed/<TICKER>_processed.csv`. Cơ chế ghép snapshot lấy theo code `build_masked_graph_manifest` và `chronological_split` (`baselines/2026-08-08_pooled_news_gnn_ablation_baseline/code/data.py`).

1. Nguyên tắc chia dữ liệu

- Chia theo thời gian (chronological), theo TỪNG mã: mỗi mã được cắt 70% train / 15% validation / 15% test **trên chính chuỗi thời gian của nó**, cắt TRƯỚC khi tạo feature / fit scaler / dựng cửa sổ.
- Vì mỗi mã có độ dài lịch sử khác nhau nên **biên ngày train/val/test khác nhau giữa các mã**.
- Scaler (chuẩn hoá) fit **chỉ trên phần train** của từng mã; cạnh đồ thị (vol→PK) ước lượng **chỉ trên train** rồi đóng băng cho val/test.
- Dự báo: giá trị Parkinson **một ngày** tại $t+h$ ($h \in \{1,5,10,22\}$), **không phải trung bình h ngày**.
- Cửa sổ đầu vào (SEQ) = 22 ngày giao dịch (cho LSTM và News).

2. Biên split thật của một số mã (tính từ dữ liệu)

Mã	Số phiên	Bắt đầu	TRAIN	VALIDATION	TEST
VNM	4887	2006-10-27	2006-10-27 → 2020-07-21	2020-07-22 → 2023-06-27	2023-06-28 → 2026-06-09
ACB	4868	2006-11-21	2006-11-21 → 2020-07-22	2020-07-23 → 2023-06-30	2023-07-03 → 2026-06-09

Mã	Số phiên	Bắt đầu	TRAIN	VALIDATION	TEST
LPB	1456	2020-11-09	2020-11-09 → 2024-10-14	2024-10-15 → 2025-08-18	2025-08-19 → 2026-06-19
SSB	1299	2021-03-24	2021-03-24 → 2024-11-08	2024-11-11 → 2025-08-21	2025-08-22 → 2026-06-09

Nhận xét: cùng một ngày lịch có thể rơi vào **các split khác nhau tùy mã**. Ví dụ ngày **2024-06-01**: - VNM, ACB: đã ở **TEST** (test bắt đầu 2023-06-28 / 2023-07-03). - LPB, SSB: vẫn ở **TRAIN** (train kết thúc 2024-10-14 / 2024-11-08).

3. Luồng LSTM và News: mỗi mẫu là MỘT mã, MỘT cửa sổ của chính nó

Một mẫu (pooled sample) = (mã, ngày mục tiêu): - Đầu vào = **22 ngày giao dịch liền trước của CHÍNH mã đó** (giá + news + mask của mã đó). - Đích = Parkinson của mã đó tại t+h.

Ví dụ: mẫu (VNM, 2015-03-20, h=5) dùng cửa sổ giá/news của **VNM** từ ~2015-02-16 đến 2015-03-20, dự báo Parkinson của **VNM** tại 2015-03-27. Mẫu này **không liên quan** đến ngày của bất kỳ mã nào khác.

“Pooled” chỉ có nghĩa là **gộp tất cả các mẫu (mã, ngày) vào một tập train chung**; mô hình xử lý **từng mẫu độc lập**. Do đó **không có** chuyện ghép cửa sổ của VNM với cửa sổ ngày khác của ACB.

4. Luồng GNN: mỗi snapshot là MỘT ngày chung, mã vắng bị mask

Một graph snapshot = **một ngày mục tiêu target_date duy nhất**, các node là các mã. Quy tắc (code build_masked_graph_manifest, GraphSnapshot): 1. **Mọi node trong một snapshot có cùng target_date** (cùng một ngày lịch). Không thể có node ngày khác trong cùng snapshot. 2. Mã **không có cửa sổ kết thúc đúng ngày đó** → presence_mask = 0 (bị **mask**, KHÔNG điền giá trị, KHÔNG dời sang ngày khác). 3. **Nhóm snapshot diễn ra BÊN TRONG từng split**: một snapshot train chỉ chứa các mã mà ngày đó thuộc train của chính mã đó (ràng buộc “mọi node cùng split” được kiểm tra cứng trong code). 4. Message passing / loss / metric chỉ chạy trên **node có mặt** (present).

Ví dụ A — ngày 2024-06-01 (cùng ngày, khác split theo mã)

Theo bảng §2, ngày 2024-06-01: SSB, LPB đang **train**; VNM, ACB đã **test**.

- **Snapshot TRAIN cho 2024-06-01**: node có mặt = {SSB, LPB, ... các mã còn ở train ngày này}. VNM, ACB **không** có mặt trong snapshot train này (chúng đã ở test) → presence=0.
- **Snapshot TEST cho 2024-06-01**: node có mặt = {VNM, ACB, ... các mã đã ở test}. SSB, LPB **không** có mặt.

→ Node của VNM ngày 2024-06-01 và node của SSB ngày 2024-06-01 **không bao giờ nằm chung một snapshot train** — chúng thuộc hai snapshot (test và train) khác nhau. **Không leakage** (snapshot train không chứa node test), và mọi node trong một snapshot đều cùng một ngày 2024-06-01.

Ví dụ B — ngày 2019-01-02 (mã chưa niêm yết)

Ngày 2019-01-02: VNM, ACB đang **train**; SSB, LPB **chưa niêm yết** (bắt đầu 2020/2021).

- **Snapshot TRAIN cho 2019-01-02:** node có mặt = {VNM, ACB, ... các mã đã giao dịch}. SSB, LPB → presence=0 (mask, KHÔNG bịa dữ liệu).

Điều KHÔNG xảy ra (đúng lo ngại cần loại trừ)

- **KHÔNG** ghép ACB@2006-08-21 với VNM@2006-08-10 trong một snapshot — một snapshot chỉ có một ngày; mã vắng ngày đó bị mask, không bị thay bằng ngày khác.
- **KHÔNG** đưa node thuộc val/test của một mã vào snapshot train — nhóm theo từng split đảm bảo điều này.

5. Vì sao không lịch ngày và không leakage (tóm tắt)

Câu hỏi	Trả lời	Cơ chế / dẫn chứng
Node trong 1 snapshot có cùng ngày không?	Có	GraphSnapshot “common-date”; kiểm tra cứng mọi node cùng target_date
Mã vắng ngày đó xử lý sao?	presence_mask=0, mask ra, không impute/dời ngày	build_masked_graph_manifest
Biên split khác nhau giữa mã có gây trộn split không?	Không	Nhóm snapshot bên trong từng split ; ràng buộc “mọi node cùng split”
LSTM/News có trộn ngày chéo mã không?	Không	Mỗi mẫu = 1 mã, cửa sổ của chính nó
Biên leakage-safe của train là gì?	train_max_date = ngày target lớn nhất của split train	GraphManifest

Hệ quả (đúng thiết kế, không phải lỗi): vì chia theo từng mã và mask theo ngày, thành phần láng giềng của đồ thị thay đổi theo thời gian (ngày sớm: nhiều mã lịch sử dài; ngày muộn trong train: các mã lịch sử ngắn còn ở train). Đây là bản chất “availability-aware masked graph”.

6. Quy mô quan sát thực tế (5 feature + news + vol→PK, h=5)

- Số snapshot (theo ngày, đã mask): ~6.470.
- Số quan sát node có mặt: **train ~73.026, validation ~14.418, test ~14.464** (h=5; thay đổi nhẹ theo horizon do warm-up của số 22 ngày + độ trễ h).
- Universe: 33 mã VN30 (điểm-thời-gian cố định).

7. Tham chiếu code

- Chia theo mã: `chronological_split (data.py)`.
- Dựng snapshot masked theo ngày chung, nhóm trong từng split: `build_masked_graph_manifest (data.py)`.
- Ràng buộc mọi node cùng ngày + cùng split: lớp `GraphSnapshot (data.py)`.
- Cạnh vol→PK ước lượng train-only rồi đóng băng: `edges.build_vol2pk_adjacency (baselines/2026-08-11_eda_gnn_baseline/code/edges.py)`.